EBS vs S3: Choosing the Right Storage Option

Task 1

Create a table comparing Amazon EBS and Amazon S3, listing at least 4 key differences (such as data type, access method, use case, and scalability) and give one real-world app example where each would be the better choice.

Aspect	Amazon EBS	Amazon S3
Data type	Block storage — raw disk blocks, formatted with a file system (ext4, xfs, etc.)	Object storage — whole files/objects stored with metadata, no file system required
Access method	Attached to a single EC2 instance at a time; accessed like a normal disk/mount point over the network within one Availability Zone	Accessed over HTTPS via URL/API from anywhere, by any number of clients simultaneously
Use case	Low-latency, high-IOPS data an application reads/writes constantly, such as a database's data files or an OS boot volume	Independent files served to many users/apps at once — images, videos, backups, static website assets, logs
Scalability	Fixed size you provision upfront (can be resized, but capacity/performance is tied to one volume and one instance)	Scales automatically and virtually without limit — no need to provision capacity in advance

Real-world app examples:

• EBS would be the better choice for Zomato's order database server — the database engine needs a persistent, low-latency block volume attached to one EC2 instance to constantly read and write order/transaction data.

• S3 would be the better choice for Instagram's photo storage — millions of independent image files need to be uploaded once and served to huge numbers of users concurrently over URLs, which is exactly what object storage is built for.

Task 2

Launch a new EC2 instance in the AWS Free Tier and attach a new EBS volume of at least 2 GB to it using the AWS Management Console.

Steps followed:

1. Opened the EC2 console and clicked "Launch instance".

2. Named the instance "ebs-demo-instance", selected the "Amazon Linux 2" AMI, and chose the "t2.micro" instance type (Free Tier eligible).

3. Selected an existing key pair for SSH access and allowed SSH (port 22) in the security group.

4. Launched the instance and waited for its state to become "Running".

5. Went to EC2 → "Volumes" → "Create volume", set Size = 2 GB, Volume type = "gp3", and matched the Availability Zone to the one the new instance was running in (EBS volumes can only attach to instances in the same AZ).

6. Created the volume, then selected it, clicked "Actions" → "Attach volume", chose "ebs-demo-instance" as the target, and confirmed the device name (e.g., /dev/sdf).

7. Confirmed the volume's state changed to "In-use" and it was now listed as attached to ebs-demo-instance.

Task 3

Log in to your EC2 instance using SSH and format the attached EBS volume, then mount it to a new directory named /mnt/mydata.

Steps followed:

1. Connected to the instance from a terminal using SSH: ssh -i my-key.pem ec2-user@<public-ip>.

2. Ran lsblk to list block devices and identify the new volume, which showed up as an unformatted device (e.g., xvdf) with no mount point:

```
$ lsblk
NAME    MAJ:MIN RM SIZE RO TYPE MOUNTPOINT
xvda    202:0    0   8G  0 disk
└─xvda1 202:1    0   8G  0 part /
xvdf    202:80   0   2G  0 disk
```

3. Formatted the new volume with an ext4 file system:

```
$ sudo mkfs -t ext4 /dev/xvdf
```

4. Created the mount point directory:

```
$ sudo mkdir /mnt/mydata
```

5. Mounted the formatted volume to the new directory:

```
$ sudo mount /dev/xvdf /mnt/mydata
```

6. Verified the mount with df -h, which showed /mnt/mydata backed by /dev/xvdf with about 2 GB of space available.

7. Added an entry to /etc/fstab (using the volume's UUID from blkid) so the mount would persist automatically across instance reboots.

Task 4

Simulate a Zomato-style image upload: create a text file named zomato_menu.txt on your EC2 instance, copy it to your mounted EBS volume, and verify it is accessible from the new mount point.

Steps followed:

1. Created a sample file in the instance's home directory:

```
$ echo "Paneer Tikka - Rs.220\nVeg Biryani - Rs.180\nCold Coffee - Rs.90" >
~/zomato_menu.txt
```

2. Copied the file into the mounted EBS volume:

```
$ cp ~/zomato_menu.txt /mnt/mydata/
```

3. Listed the contents of the mount point to confirm the file was present:

```
$ ls -l /mnt/mydata
-rw-rw-r-- 1 ec2-user ec2-user 58 Aug  9 10:15 zomato_menu.txt
```

4. Read the file directly from the mounted volume to confirm it was fully accessible and intact:

```
$ cat /mnt/mydata/zomato_menu.txt
Paneer Tikka - Rs.220
Veg Biryani - Rs.180
Cold Coffee - Rs.90
```

5. This confirmed the EBS volume behaves exactly like a normal disk directory once mounted — any file copied to it is immediately readable/writable from that mount point.

Task 5

Detach the EBS volume from your EC2 instance using the AWS Console, then reattach it to the same or a different instance. Describe what happens to the data you stored and why this behavior is useful for apps like Flipkart or Myntra order tracking.

Steps followed:

1. Before detaching, ran sudo umount /mnt/mydata on the instance to safely unmount the volume from the file system.

2. In the EC2 console, went to "Volumes", selected the 2 GB volume, clicked "Actions" → "Detach volume", and confirmed.

3. Waited for the volume's state to change to "Available" (no longer attached to any instance).

4. Selected the same volume again, clicked "Actions" → "Attach volume", and reattached it to the same "ebs-demo-instance" (this would work the same way for a different instance, as long as it is in the same Availability Zone).

5. SSHed back into the instance, ran lsblk to confirm the device appeared again as /dev/xvdf, and mounted it again with sudo mount /dev/xvdf /mnt/mydata (no reformatting needed this time).

6. Ran cat /mnt/mydata/zomato_menu.txt again and confirmed the exact same menu content was still there, untouched by the detach/reattach cycle.

What happens to the data:

The data stored on the EBS volume is completely preserved through detach and reattach — EBS volumes exist independently of any single EC2 instance, so unmounting and detaching does not delete or reset anything on the volume. The same block device, with the same file system and files, simply becomes available to be mounted on another instance.

Why this is useful for apps like Flipkart or Myntra:

This durability and portability is exactly what order-tracking systems rely on. For example, if the EC2 instance running Flipkart's order-processing service crashes or needs to be replaced, the EBS volume holding order data can be detached from the failed instance and reattached to a fresh, healthy instance without losing any order records — the new instance picks up right where the old one left off. It also means maintenance, instance-type upgrades, or migrating a service to a different server can be done by simply moving the volume, rather than having to re-copy or risk losing critical transactional data like order history and shipment status.